

```

1 import mysql.connector as pymysql
2 from mysql.connector import Error
3 from tabulate import tabulate
4
5 host = "localhost"
6 root_user = "root"
7 root_password = "12345678"
8 database = "hospital_db"
9
10 def connect_root():          # Connects to MySQL as root
11     try:
12         return pymysql.connect(host=host, user=root_user, password=
root_password)
13     except Error as e:
14         print("\033[1;31mCould not connect to MySQL server:\033[0m", e)
15         exit()
16
17 def connect_db():           # Tries to connect to Database
18     try:
19         return pymysql.connect(host=host, user=root_user, password=
root_password, database=database)
20     except Error as e:
21         print(f"\033[1;31mCould not connect to database {database}:\033[0m",
e)
22         exit()
23
24 def db_init():              # Set-up database with tables
25     root = connect_root()
26     cur_root = root.cursor()
27     try:
28         print("\033[0;36mEnsuring database exists...\033[0m")
29         cur_root.execute(f"CREATE DATABASE IF NOT EXISTS `{database}`;")
30         cur_root.execute(f"USE `{database}`;")
31     except Error as e:
32         print("\033[1;31mDatabase initialisation failed:\033[0m", e)
33         root.close()
34         exit()
35     root.close()
36     con = connect_db()
37     cur = con.cursor()
38     try:
39         # USERS table
40         cur.execute("""
41             CREATE TABLE IF NOT EXISTS USERS (
42                 uid INT AUTO_INCREMENT PRIMARY KEY,
43                 username VARCHAR(30) UNIQUE,
44                 password VARCHAR(128) NOT NULL,
45                 name VARCHAR(30) NOT NULL,
46                 role ENUM('admin','doctor','pharmacist','patient') NOT NULL,
47                 specialization VARCHAR(100) DEFAULT NULL );
48         """)
49         # PHARMACY table
50         cur.execute("""
51             CREATE TABLE IF NOT EXISTS pharmacy (
52                 med_id INT AUTO_INCREMENT PRIMARY KEY,
53                 med_name VARCHAR(50) NOT NULL,
54                 quantity INT NOT NULL DEFAULT 0,
55                 price DECIMAL(10,2) NOT NULL );
56         """)
57         # SALES_LOG table
58         cur.execute("""
59             CREATE TABLE IF NOT EXISTS sales_log (

```

```

60         sale_id INT AUTO_INCREMENT PRIMARY KEY,
61         med_id INT NOT NULL,
62         quantity_sold INT NOT NULL,
63         total_price DECIMAL(10,2) NOT NULL,
64         sold_by INT,
65         sold_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
66         FOREIGN KEY (med_id) REFERENCES pharmacy(med_id)
67             ON DELETE CASCADE ON UPDATE CASCADE,
68         FOREIGN KEY (sold_by) REFERENCES USERS(uid)
69             ON DELETE SET NULL ON UPDATE CASCADE );
70     """
71     # PRESCRIPTIONS TABLE
72     cur.execute("""
73         CREATE TABLE IF NOT EXISTS prescriptions (
74             prescription_id INT AUTO_INCREMENT PRIMARY KEY,
75             doctor_uid INT NOT NULL,
76             patient_uid INT NOT NULL,
77             description TEXT NOT NULL,
78             date_issued DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
79             patient_name VARCHAR(50),
80             patient_age INT,
81             patient_gender VARCHAR(20),
82             patient_phone VARCHAR(14),
83             disease VARCHAR(128),
84             FOREIGN KEY (doctor_uid) REFERENCES USERS(uid)
85                 ON DELETE CASCADE ON UPDATE CASCADE,
86             FOREIGN KEY (patient_uid) REFERENCES USERS(uid)
87                 ON DELETE CASCADE ON UPDATE CASCADE);
88     """)
89     # BEDS table for maintaining beds
90     cur.execute("""
91         CREATE TABLE IF NOT EXISTS beds (
92             bed_id INT AUTO_INCREMENT PRIMARY KEY,
93             status ENUM('Available','Occupied') NOT NULL DEFAULT '
Available',
94             patient_uid INT DEFAULT NULL,
95             FOREIGN KEY (patient_uid) REFERENCES USERS(uid)
96                 ON DELETE SET NULL ON UPDATE CASCADE );
97     """)
98     con.commit()
99
100     # Ensure at least one admin exists, else create one
101     cur_admin = con.cursor(dictionary=True)
102     try:
103         cur_admin.execute("SELECT COUNT(*) AS cnt FROM USERS WHERE role
='admin';")
104         r = cur_admin.fetchone()
105         if r:
106             cnt = r['cnt']
107         else:
108             cnt = 0
109         if cnt == 0:
110             try:
111                 cur_admin.execute(
112                     "INSERT INTO USERS (username, password, name, role,
specialization) VALUES (%s, %s, %s, %s, %s)",
113                     ("superuser000", "admin123457", "default_admin_0", "
admin", None))
114                 con.commit()
115                 print("\033[1;32mDefault admin 'superuser000' created (
password 'admin123457').\033[0m")
116             except Error as e:

```

```

117         print("\033[1;31mCould not create default admin:\033[0m"
, e)
118         finally:
119             cur_admin.close()
120             print("\033[1;32mDatabase initialization complete.\033[0m")
121     except Error as e:
122         print("\033[1;31mInitialization SQL error:\033[0m", e)
123     finally:
124         cur.close()
125         con.close()
126
127 def login():                                     #Login function for authenticating
users
128     con = connect_db()
129     cur = con.cursor(dictionary=True)
130     try:
131         print("\033[0;36m=== LOGIN ===\033[0m")
132         username = input("Username: ")
133         password = input("Password: ")
134         cur.execute("SELECT * FROM USERS WHERE username=%s AND password=%s"
, (username, password))
135         user = cur.fetchone()
136         if user:
137             print(f"\033[1;32mLogged in as {user['name']},{user['role']}\033
[0m")
138             return user
139         else:
140             print("\033[1;31mInvalid credentials.\033[0m")
141             return None
142     except Error as e:
143         print("\033[1;31mLogin error:\033[0m", e)
144         return None
145     finally:
146         cur.close()
147         con.close()
148
149 def change_password_user(uid):                   #Change user account
password
150     con = connect_db()
151     cur1 = con.cursor(dictionary=True)
152     try:
153         print("\033[0;36mCHANGE PASSWORD\033[0m")
154         old = input("Current password: ")
155         cur1.execute("SELECT password FROM USERS WHERE uid=%s", (uid,))
156         row = cur1.fetchone()
157         if not row:
158             print("\033[1;31mUser not found.\033[0m")
159             return
160         db_pass = row['password']
161         if old != db_pass:
162             print("\033[1;31mCurrent password incorrect.\033[0m")
163             return
164         new = input("New password: ")
165         confirm = input("Confirm new password: ")
166         if new != confirm:
167             print("\033[1;31mPasswords do not match.\033[0m")
168             return
169         try:
170             cur1.execute("UPDATE USERS SET password=%s WHERE uid=%s", (new,
uid))
171             con.commit()
172             print("\033[1;32mPassword changed successfully.\033[0m")

```

```

173         except Error as e:
174             print("\033[1;31mFailed to change password:\033[0m", e)
175     finally:
176         cur1.close()
177         con.close()
178
179 def admin_create_employee():                                #Creates entry for doctor or
    pharmacist
180     con = connect_db()
181     cur = con.cursor()
182     try:
183         print("\033[0;36mCreate new employee (doctor or pharmacist)\033[0m")
184         name = input("Name: ")
185         username = input("Username: ")
186         password = input("Password: ")
187         while True:
188             ch = input("Role (1. doctor/ 2. pharmacist): ")
189             if ch not in ("1", "2"):
190                 print("\033[1;31mChoose 1 or 2: \033[0m")
191                 continue
192             else:
193                 role = "doctor" if ch == "1" else "pharmacist"
194                 break
195             specialization = None
196             if role == 'doctor':
197                 spec_list = ["General Medicine", "Cardiology", "Neurology", "
Orthopedics", "Pediatrics",
198                             "Dermatology", "ENT", "Gynecology", "Nephrology"]
199                 for i in range(1, len(spec_list) + 1):
200                     print(f"{i}. {spec_list[i - 1]}")                    #Prints as <
1. spec>
201                 try:
202                     ch = int(input("Choose: "))
203                     if 1 <= ch <= len(spec_list):
204                         specialization = spec_list[ch - 1]
205                     else:
206                         specialization = None
207                 except ValueError:
208                     specialization = None
209                 try:
210                     cur.execute(
211                         "INSERT INTO USERS (username, password, name, role,
specialization) VALUES (%s,%s,%s,%s,%s)",
212                         (username, password, name, role, specialization))
213                     con.commit()
214                     print(f"\033[1;32mEmployee created with uid {cur.lastrowid}\033[
0m")
215                 except Error as e:
216                     print("\033[1;31mCould not create employee:\033[0m", e)
217             finally:
218                 cur.close()
219                 con.close()
220
221 def admin_remove_employee(user):                            #Can remove doctor
    or pharmacist
222     con = connect_db()
223     cur = con.cursor(dictionary=True)
224     try:
225         print("\033[0;36mRemove employee:\033[0m")
226         try:
227             uid = int(input("Enter employee uid to remove: "))
228             except ValueError:

```



```

229         print("\033[1;31mInvalid uid.\033[0m")
230         return
231     if uid == 1:
232         print("\033[1;31mCannot remove default admin uid 1.\033[0m")
233         return
234     cur.execute("SELECT * FROM USERS WHERE uid=%s", (uid,))
235     row = cur.fetchone()
236     if not row:
237         print("\033[1;31mNot found.\033[0m")
238         return
239     role = row['role']
240     if role == 'admin':
241         print("\033[1;31mAdmins cannot remove other admins.\033[0m")
242         return
243     print(f"\033[1;33mYou are about to REMOVE '{row['username']}' (role
={row['role']}).\033[0m")
244     confirm = input("Are you sure? (yes/no): ")
245     if confirm != "yes":
246         print("\033[1;31mRemoval cancelled.\033[0m")
247         return
248     admin_pass = input("Enter your admin password to confirm: ")
249     cur.execute("SELECT password FROM USERS WHERE uid=%s", (user['uid'
],))
250     row2 = cur.fetchone()
251     if not row2 or admin_pass != row2['password']:
252         print("\033[1;31mPassword incorrect. Cannot proceed.\033[0m")
253         return
254     try:
255         cur.execute("DELETE FROM USERS WHERE uid=%s", (uid,))
256         con.commit()
257         print("\033[1;32mEmployee removed successfully.\033[0m")
258     except Error as e:
259         print("\033[1;31mRemove failed:\033[0m", e)
260     finally:
261         cur.close()
262         con.close()
263
264 def admin_modify_stock():                                #Enter records of
new med or update them
265     con = connect_db()
266     cur = con.cursor()
267     try:
268         print("\033[0;36mModify stock (add new med or update existing)\033[
0m")
269         print("1. New")
270         print("2. Update")
271         ch = input("\033[1;33mChoice: \033[0m")
272         if ch == '1':
273             name = input("Medicine name: ")
274             try:
275                 qty = int(input("Quantity (integer): "))
276                 price = float(input("Price per unit: "))
277             except ValueError:
278                 print("\033[1;31mInvalid numeric input.\033[0m")
279                 return
280             try:
281                 cur2 = con.cursor()
282                 cur2.execute("INSERT INTO pharmacy (med_name, quantity,
price) VALUES (%s,%s,%s)",
(name, qty, price))
283                 con.commit()
284                 cur2.close()
285

```

```

286         print("\033[1;32mMedicine added.\033[0m")
287     except Error as e:
288
289         print("\033[1;31mInsert failed:\033[0m", e)
290     elif ch == '2':
291         try:
292             mid = int(input("med_id: "))
293             cur.execute("SELECT * FROM pharmacy WHERE med_id=%s", (mid
294 ,))
295             med = cur.fetchone()
296             if not med:
297                 print("\033[1;31mMedicine not found.\033[0m")
298                 return
299             print("Current:", med)
300             qty = input("New quantity (leave blank to skip): ")
301             if qty != "":
302                 try:
303                     qv = int(qty)
304                     cur.execute("UPDATE pharmacy SET quantity=%s WHERE
305 med_id=%s", (qv, mid))
306                     except ValueError:
307                         print("\033[1;31mInvalid quantity.\033[0m")
308                         return
309                     price = input("New price (leave blank to skip): ")
310                     if price != "":
311                         try:
312                             pv = float(price)
313                             cur.execute("UPDATE pharmacy SET price=%s WHERE
314 med_id=%s", (pv, mid))
315                     except ValueError:
316                         print("\033[1;31mInvalid price.\033[0m")
317                         return
318                     con.commit()
319                     print("\033[1;32mUpdate successful.\033[0m")
320                     except ValueError:
321                         print("\033[1;31mInvalid med_id.\033[0m")
322                     else:
323                         print("\033[1;31mInvalid option.\033[0m")
324                 finally:
325                     cur.close()
326                     con.close()
327
328 def admin_view_stock():
329     #View available
330     medicine stock
331     try:
332         con = connect_db()
333         cur = con.cursor(dictionary=True)
334         cur.execute("SELECT * FROM pharmacy ORDER BY med_name")
335         rows = cur.fetchall()
336         cur.close()
337         con.close()
338     except Error as e:
339         print("\033[1;31mDB error:\033[0m", e)
340         return
341     if not rows:
342         print("\033[0;33mNo medicines found.\033[0m")
343         return
344     print(tabulate(rows, headers="keys", tablefmt="grid"))
345
346 def admin_view_sales_log():
347     #View previous sale
348     records
349     try:

```

```

343         con = connect_db()
344         cur = con.cursor(dictionary=True)
345         cur.execute("""
346             SELECT s.sale_id, p.med_name, s.quantity_sold, s.total_price, u.
name AS sold_by, s.sold_at
347             FROM sales_log s, pharmacy p, users u
348             where s.sold_by = u.uid
349             AND s.med_id = p.med_id
350             ORDER BY s.sold_at DESC
351         """)
352         rows = cur.fetchall()
353         cur.close()
354         con.close()
355     except Error as e:
356         print("\033[1;31mDB error:\033[0m", e)
357         return
358     if not rows:
359         print("\033[0;33mNo sales recorded.\033[0m")
360         return
361     print(tabulate(rows, headers="keys", tablefmt="grid"))
362
363 def admin_add_bed():                                #Add new beds
364     con = connect_db()
365     cur = con.cursor()
366     try:
367         count = input("How many beds to add (integer): ")
368         try:
369             count = int(count)
370         except ValueError:
371             print("\033[1;31mInvalid integer.\033[0m")
372             return
373         try:
374             for i in range(count):
375                 cur.execute("INSERT INTO beds (status, patient_uid) VALUES
('Available', NULL)")
376                 con.commit()
377                 print(f"\033[1;32mAdded {count} beds.\033[0m")
378             except Error as e:
379                 print("\033[1;31mOperation Failed! Reason:\033[0m", e)
380         finally:
381             cur.close()
382             con.close()
383
384 def admin_remove_bed():                            #Remove bed
385     con = connect_db()
386     cur = con.cursor(dictionary=True)
387     try:
388         try:
389             bed_id = int(input("Enter bed_id to remove: "))
390         except ValueError:
391             print("\033[1;31mInvalid bed_id.\033[0m")
392             return
393         cur.execute("SELECT * FROM beds WHERE bed_id=%s", (bed_id,))
394         b = cur.fetchone()
395         if not b:
396             print("\033[1;31mBed not found.\033[0m")
397             return
398         status = b['status']
399         if status != 'Available':
400             print("\033[1;31mCannot remove occupied bed.\033[0m")
401             return
402         try:

```

```

403         cur.execute("DELETE FROM beds WHERE bed_id=%s", (bed_id,))
404         con.commit()
405         print("\033[1;32mBed removed.\033[0m")
406     except Error as e:
407         print("\033[1;31mRemove failed:\033[0m", e)
408     finally:
409         cur.close()
410         con.close()
411
412 def admin_view_beds():                                     #View current bed
    status
413     try:
414         con = connect_db()
415         cur = con.cursor(dictionary=True)
416         cur.execute("SELECT * FROM beds ORDER BY bed_id")
417         rows = cur.fetchall()
418         cur.close()
419         con.close()
420     except Error as e:
421         print("\033[1;31mDB error:\033[0m", e)
422         return
423     if not rows:
424         print("\033[0;33mNo beds in system.\033[0m")
425         return
426     print(tabulate(rows, headers="keys", tablefmt="grid"))
427
428
429 def admin_view_all_prescriptions():                       #View all
    prescriptions in system
430     try:
431         con = connect_db()
432         cur = con.cursor(dictionary=True)
433         cur.execute("""
434             SELECT pr.prescription_id as p_id,
435                    u_doctor.name AS doc,
436                    u_patient.name AS patient,
437                    u_patient.uid as pat_uid,
438                    pr.description,
439                    pr.date_issued as DATE
440             FROM prescriptions pr, USERS u_doctor, USERS u_patient
441             WHERE pr.doctor_uid = u_doctor.uid
442             AND pr.patient_uid = u_patient.uid
443             ORDER BY pr.date_issued DESC;
444         """)
445         rows = cur.fetchall()
446         cur.close()
447         con.close()
448     except Error as e:
449         print("\033[1;31mDB error:\033[0m", e)
450         return
451     if not rows:
452         print("\033[0;33mNo prescriptions found.\033[0m")
453         return
454     print(tabulate(rows, headers="keys", tablefmt="grid"))
455
456
457 def doctor_view_patients():                               #View patient list
458     try:
459         con = connect_db()
460         cur = con.cursor(dictionary=True)
461         cur.execute("SELECT uid, name FROM USERS WHERE role='patient' ORDER
    BY name")

```

```

462         rows = cur.fetchall()
463         cur.close()
464         con.close()
465     except Error as e:
466         print("\033[1;31mDB error:\033[0m", e)
467         return
468     if not rows:
469         print("\033[0;33mNo patients found.\033[0m")
470         return
471     print(tabulate(rows, headers="keys", tablefmt="grid"))
472
473
474 def doctor_create_prescription(user):                                #Create prescription
475     con = connect_db()
476     cur = con.cursor(dictionary=True)
477     try:
478         try:
479             patient_uid = int(input("Patient uid: "))
480         except ValueError:
481             print("\033[1;31mInvalid patient uid.\033[0m")
482             return
483         cur.execute("SELECT * FROM USERS WHERE uid=%s AND role='patient'", (
patient_uid,))
484         row = cur.fetchone()
485         if not row:
486             print("\033[1;31mPatient not found.\033[0m")
487             return
488         else:
489             print(f"\033[1;32mPatient found: {row['name']}, uid: {row['uid']}
\033[0m")
490         print("Enter prescription text. End with a blank line.")
491         lines = []
492         while True:
493             line = input()
494             if line == "":
495                 break
496             lines.append(line)
497         text = "\n".join(lines)
498         if not text:
499             print("\033[1;31mPrescription cannot be empty.\033[0m")
500             return
501         try:
502             patient_name = input("Patient name: ")
503         except:
504             patient_age = int(input("Patient age: "))
505             except ValueError:
506                 patient_age = None
507             patient_gender = input("Patient gender: ")
508             patient_phone = input("Patient phone: ")
509             disease = input("Disease / Diagnosis: ")
510             cur.execute(
511                 """
512                 INSERT INTO prescriptions
513                 (doctor_uid, patient_uid, description, date_issued,
514                 patient_name, patient_age, patient_gender, patient_phone,
disease)
515                 VALUES (%s, %s, %s, NOW(), %s, %s, %s, %s, %s)
516                 """
517                 (user["uid"], patient_uid, text, patient_name, patient_age,
patient_gender, patient_phone, disease)
518             )
519             con.commit()

```

```

520         print(f"\033[1;32mPrescription created (id {cur.lastrowid}).\033
[0m")
521     except Error as e:
522         print("\033[1;31mInsert failed:\033[0m", e)
523     finally:
524         cur.close()
525         con.close()
526
527
528 def doctor_view_patient_prescriptions():                #View specific
    patient prescription
529     con = connect_db()
530     cur = con.cursor(dictionary=True)
531     try:
532         try:
533             patient_uid = int(input("Patient uid to view: "))
534         except ValueError:
535             print("\033[1;31mInvalid uid.\033[0m")
536             return
537         cur.execute("""
538             SELECT p.prescription_id,
539                    u_doctor.name AS doctor,
540                    p.description,
541                    p.date_issued
542             FROM prescriptions p,
543                    USERS u_doctor
544             WHERE p.doctor_uid = u_doctor.uid
545             AND p.patient_uid = %s
546             ORDER BY p.date_issued DESC;""", (patient_uid,))
547         rows = cur.fetchall()
548         if not rows:
549             print("\033[0;33mNo prescriptions for this patient.\033[0m")
550             return
551         print(tabulate(rows, headers="keys", tablefmt="grid"))
552     except Error as e:
553         print("\033[1;31mDB error:\033[0m", e)
554     finally:
555         cur.close()
556         con.close()
557
558
559 def doctor_assign_bed():                                #Assign bed to
    patient if needed
560     con = connect_db()
561     cur = con.cursor(dictionary=True)
562     try:
563         try:
564             patient_uid = int(input("Patient uid to assign bed: "))
565         except ValueError:
566             print("\033[1;31mInvalid uid.\033[0m")
567             return
568         cur.execute("SELECT * FROM USERS WHERE uid=%s AND role='patient'", (
patient_uid,))
569         if not cur.fetchone():
570             print("\033[1;31mPatient not found.\033[0m")
571             return
572         cur.execute(
573             "SELECT * FROM beds WHERE patient_uid=%s AND status='Occupied'",
574             (patient_uid,)
575         )
576         existing = cur.fetchone()
577         if existing:

```

```

578         print(f"\033[1;31mPatient already occupies bed {existing['bed_id']}\033[0m")
579         return
580         cur.execute("SELECT * FROM beds WHERE status='Available' LIMIT 1")
581         bed = cur.fetchone()
582         if not bed:
583             print("\033[1;31mNo available beds.\033[0m")
584             return
585         try:
586             cur.execute("UPDATE beds SET status='Occupied', patient_uid=%s
WHERE bed_id=%s",
587                         (patient_uid, bed['bed_id']))
588             con.commit()
589             print(f"\033[1;32mAssigned bed {bed['bed_id']} to patient {
patient_uid}\033[0m")
590         except Error as e:
591             print("\033[1;31mAssign failed:\033[0m", e)
592         finally:
593             cur.close()
594             con.close()
595
596
597 def doctor_release_bed():                                #Discharge patient,
release bed
598     con = connect_db()
599     cur = con.cursor(dictionary=True)
600     try:
601         try:
602             patient_uid = int(input("Patient uid to discharge (release bed
): "))
603         except ValueError:
604             print("\033[1;31mInvalid uid.\033[0m")
605             return
606         cur.execute("SELECT * FROM beds WHERE patient_uid=%s AND status='
Occupied' LIMIT 1", (patient_uid,))
607         bed = cur.fetchone()
608         if not bed:
609             print("\033[0;33mNo bed assigned to this patient.\033[0m")
610             return
611         try:
612             cur.execute("UPDATE beds SET status='Available', patient_uid=
NULL WHERE bed_id=%s", (bed['bed_id'],))
613             con.commit()
614             print(f"\033[1;32mBed {bed['bed_id']} released.\033[0m")
615         except Error as e:
616             print("\033[1;31mRelease failed:\033[0m", e)
617         finally:
618             cur.close()
619             con.close()
620
621
622 def pharmacist_view_stock():                                #View medicine stock
623     try:
624         con = connect_db()
625         cur = con.cursor(dictionary=True)
626         cur.execute("SELECT * FROM pharmacy ORDER BY med_id")
627         rows = cur.fetchall()
628         cur.close()
629         con.close()
630     except Error as e:
631         print("\033[1;31mDB error:\033[0m", e)
632     return

```

```

633     if not rows:
634         print("\033[0;33mNo stock found.\033[0m")
635         return
636     print(tabulate(rows, headers="keys", tablefmt="grid"))
637
638
639 def pharmacist_update_stock():                #Update Available stock
640     con = connect_db()
641     cur = con.cursor()
642     try:
643         try:
644             med_id = int(input("med_id to update: "))
645         except ValueError:
646             print("\033[1;31mInvalid med_id.\033[0m")
647             return
648         cur.execute("SELECT * FROM pharmacy WHERE med_id=%s", (med_id,))
649         med = cur.fetchone()
650         if not med:
651             print("\033[1;31mMedicine not found.\033[0m")
652             return
653         try:
654             qty = int(input("New quantity (integer): "))
655         except ValueError:
656             print("\033[1;31mInvalid quantity.\033[0m")
657             return
658         try:
659             price = float(input("New price (float): "))
660         except ValueError:
661             print("\033[1;31mInvalid price.\033[0m")
662             return
663         try:
664             cur.execute("UPDATE pharmacy SET quantity=%s, price=%s WHERE
med_id=%s", (qty, price, med_id))
665             con.commit()
666             print("\033[1;32mStock updated.\033[0m")
667         except Error as e:
668             print("\033[1;31mUpdate failed:\033[0m", e)
669     finally:
670         cur.close()
671         con.close()
672
673
674 def pharmacist_sell_cart(user):                #Sale and billing - pharmacy
675     con = connect_db()
676     cur = con.cursor(dictionary=True)
677     cart = []
678     try:
679         while True:
680             med_input = input("Enter med_id to add to cart (or blank to
checkout): ")
681             if med_input == "":
682                 confirm = input("Confirm checkout? (y/n): ")
683                 if confirm in "Yy":
684                     break
685                 else:
686                     continue
687             try:
688                 med_id = int(med_input)
689             except ValueError:
690                 print("\033[1;31mInvalid med_id.\033[0m")
691                 continue
692             cur.execute("SELECT * FROM pharmacy WHERE med_id=%s", (med_id,))

```



```

693         med = cur.fetchone()
694         if not med:
695             print("\033[1;31mMedicine not found.\033[0m")
696             continue
697         print("Selected:", med)
698         try:
699             qty = int(input("Quantity: "))
700         except ValueError:
701             print("\033[1;31mInvalid quantity.\033[0m")
702             continue
703         if qty <= 0:
704             print("\033[1;31mQuantity must be > 0.\033[0m")
705             continue
706         available = med['quantity']
707         unit_price = float(med['price'])
708         med_name = med['med_name']
709         if qty > available:
710             print(f"\033[1;31mInsufficient stock (available {available
711             })\033[0m")
712             continue
713         cart.append((med_id, med_name, qty, unit_price))
714         print("\033[1;32mAdded to cart.\033[0m")
715     if not cart:
716         print("\033[0;33mCart empty.\033[0m")
717         return
718     print("\033[0;36m=== BILL ===\033[0m")
719     table = []
720     total = 0
721     for med_id, name, qty, price in cart:
722         item_total = qty * price
723         total += item_total
724         table.append([med_id, name, qty, price, item_total])
725     print(tabulate(
726         table,
727         headers=["Med ID", "Name", "Qty", "Unit Price", "Item Total"],
728         tablefmt="grid"
729     ))
730
731     print(f"\033[1;33mTOTAL:\033[0m {total}")
732     confirm = input("Proceed to billing? (y/n): ")
733     if confirm not in "Yy":
734         print("\033[0;33mCheckout cancelled.\033[0m")
735         return
736     try:
737         for med_id, mname, qty, price in cart:
738             line_total = qty * price
739             cur2 = con.cursor()
740             cur2.execute(
741                 "INSERT INTO sales_log (med_id, quantity_sold,
total_price, sold_by) "
742                 "VALUES (%s, %s, %s, %s)",
743                 (med_id, qty, line_total, user['uid'])
744             )
745             cur2.execute(
746                 "UPDATE pharmacy SET quantity = quantity - %s WHERE
med_id=%s",
747                 (qty, med_id)
748             )
749             cur2.close()
750             con.commit()
751     print(f"\033[1;32mCheckout successful. Total: {total}\033[0m")

```

```

752         except Error as e:
753             print("\033[1;31mCheckout failed:\033[0m", e)
754     finally:
755         cur.close()
756         con.close()
757
758 def pharmacist_view_sales_log():                #View all previous sales
759     try:
760         con = connect_db()
761         cur = con.cursor(dictionary=True)
762         cur.execute("""
763             SELECT s.sale_id, p.med_name, s.quantity_sold, s.total_price, u.
name AS sold_by, s.sold_at
764             FROM sales_log s, pharmacy p, USERS u
765             WHERE s.med_id = p.med_id
766             AND s.sold_by = u.uid
767             ORDER BY s.sold_at DESC;
768         """)
769         rows = cur.fetchall()
770         cur.close()
771         con.close()
772     except Error as e:
773         print("\033[1;31mDB error:\033[0m", e)
774         return
775     if not rows:
776         print("\033[0;33mNo sales recorded.\033[0m")
777         return
778     print(tabulate(rows, headers="keys", tablefmt="grid"))
779
780 def register_as_patient():                    #Create patient user account
781     print("\033[0;36m=== PATIENT REGISTRATION ===\033[0m")
782     con = None
783     cur = None
784     try:
785         name = input("Name: ")
786         username = input("Choose a username: ")
787         password = input("Choose a password: ")
788         if username == "" or password == "":
789             print("\033[1;31mFields cannot be empty.\033[0m")
790             return
791         con = connect_db()
792         cur = con.cursor(dictionary=True)
793         cur.execute("SELECT * FROM USERS WHERE username = %s", (username,))
794         if cur.fetchone():
795             print("\033[1;31mUsername already exists.\033[0m")
796             return
797         cur.execute("""
798             INSERT INTO USERS (username, password, name, role,
specialization)
799             VALUES (%s, %s, %s, 'patient', NULL)
800             """, (username, password, name))
801         con.commit()
802         print("\033[1;32mPatient registered successfully.\033[0m")
803     except Error as e:
804         print("\033[1;31mError registering patient:\033[0m", e)
805     finally:
806         cur.close()
807         con.close()
808
809 def patient_view_prescriptions(user):        #View own prescriptions
810     con = connect_db()
811     cur = con.cursor(dictionary=True)

```

```

812     try:
813         cur.execute("""
814             SELECT p.prescription_id, u_doctor.name AS doctor, p.description
, p.date_issued
815             FROM prescriptions p, USERS u_doctor
816             WHERE p.doctor_uid = u_doctor.uid
817             AND p.patient_uid = %s
818             ORDER BY p.date_issued DESC
819             """, (user['uid'],))
820         rows = cur.fetchall()
821         if not rows:
822             print("\033[0;33mNo prescriptions found.\033[0m")
823             return
824         print(tabulate(rows, headers="keys", tablefmt="grid"))
825     except Error as e:
826         print("\033[1;31mDB error:\033[0m", e)
827     finally:
828         cur.close()
829         con.close()
830
831
832 #MENUS
833 def admin_menu(user):
834     while True:
835         print("\033[0;36m=== ADMIN MENU ===\033[0m")
836         print("1. Create employee (doctor/pharmacist)")
837         print("2. Remove employee")
838         print("3. Modify pharmacy stock (add/update)")
839         print("4. View pharmacy stock")
840         print("5. View sales log")
841         print("6. Add beds")
842         print("7. Remove bed")
843         print("8. View beds")
844         print("9. View all prescriptions")
845         print("10. Change account password")
846         print("0. Logout")
847         choice = input("Choice: ")
848         if choice == "1":
849             admin_create_employee()
850         elif choice == "2":
851             admin_remove_employee(user)
852         elif choice == "3":
853             admin_modify_stock()
854         elif choice == "4":
855             admin_view_stock()
856         elif choice == "5":
857             admin_view_sales_log()
858         elif choice == "6":
859             admin_add_bed()
860         elif choice == "7":
861             admin_remove_bed()
862         elif choice == "8":
863             admin_view_beds()
864         elif choice == "9":
865             admin_view_all_prescriptions()
866         elif choice == "10":
867             change_password_user(user["uid"])
868         elif choice == "0":
869             break
870         else:
871             print("\033[1;31mInvalid option.\033[0m")
872

```

```

873 def doctor_menu(user):
874     while True:
875         print("\033[0;36m=== DOCTOR MENU ===\033[0m")
876         print("1. View patient list")
877         print("2. Create prescription")
878         print("3. View past prescriptions for a patient")
879         print("4. Assign bed to patient")
880         print("5. Release bed (discharge)")
881         print("6. Change account password")
882         print("0. Logout")
883         choice = input("Choice: ")
884         if choice == "1":
885             doctor_view_patients()
886         elif choice == "2":
887             doctor_create_prescription(user)
888         elif choice == "3":
889             doctor_view_patient_prescriptions()
890         elif choice == "4":
891             doctor_assign_bed()
892         elif choice == "5":
893             doctor_release_bed()
894         elif choice == "6":
895             change_password_user(user["uid"])
896         elif choice == "0":
897             break
898         else:
899             print("\033[1;31mInvalid option.\033[0m")
900
901 def pharmacist_menu(user):
902     while True:
903         print("\033[0;36m=== PHARMACIST MENU ===\033[0m")
904         print("1. View stock")
905         print("2. Update stock")
906         print("3. Sell")
907         print("4. View sales log")
908         print("5. Change account password")
909         print("0. Logout")
910         choice = input("Choice: ")
911         if choice == "1":
912             pharmacist_view_stock()
913         elif choice == "2":
914             pharmacist_update_stock()
915         elif choice == "3":
916             pharmacist_sell_cart(user)
917         elif choice == "4":
918             pharmacist_view_sales_log()
919         elif choice == "5":
920             change_password_user(user["uid"])
921         elif choice == "0":
922             break
923         else:
924             print("\033[1;31mInvalid option.\033[0m")
925
926 def patient_menu(user):
927     while True:
928         print("\033[0;36m=== PATIENT MENU ===\033[0m")
929         print("1. View my prescriptions")
930         print("2. Change account password")
931         print("0. Logout")
932         choice = input("Choice: ")
933         if choice == "1":
934             patient_view_prescriptions(user)

```

```

935         elif choice == "2":
936             change_password_user(user["uid"])
937         elif choice == "0":
938             break
939         else:
940             print("\033[1;31mInvalid option.\033[0m")
941
942     def menu():
943         db_init()
944         print("\033[1;32mWelcome to the Hospital-Management-System (v1.0)\033[0m")
945
946         while True:
947             print("\033[0;36m=== MENU ===\033[0m")
948             print("1. Login")
949             print("2. Register as Patient")
950             print("3. Exit")
951             choice = input("Choice: ")
952             if choice == "1":
953                 user = login()
954                 if not user:
955                     continue
956                 role = user['role']
957                 if role == 'admin':
958                     admin_menu(user)
959                 elif role == 'doctor':
960                     doctor_menu(user)
961                 elif role == 'pharmacist':
962                     pharmacist_menu(user)
963                 elif role == 'patient':
964                     patient_menu(user)
965                 else:
966                     print("\033[1;31mUnknown role.\033[0m")
967             elif choice == "2":
968                 register_as_patient()
969             elif choice == "3":
970                 print("\033[0;36mExiting...\033[0m")
971                 break
972             else:
973                 print("\033[1;31mInvalid option.\033[0m")
974
975     #RUN PROGRAM
976     menu()

```